
OPENCV DESDE PYTHON:

ESTEREOVISIÓN – PROYECCIÓN INVERSA

INTRODUCCIÓN

En el segundo proyecto de la asignatura se debe posicionar, con respecto a un sistema 3D de referencia del mundo, los puntos de interés (o centros) de los objetos reales. En este caso, el posicionamiento se resolverá de la manera más precisa, aplicando estereovisión con la mejor configuración de cámaras posible. En el visor virtual, deberá mostrar de manera continua la escena virtual actualizando la posición los objetos reales en movimiento según las posiciones estimadas. Además, se deberá proyectar los objetos y/o robots virtuales sobre las imágenes capturadas por una cámara generando realidad aumentada.

OBJETIVOS

El objetivo principal de esta práctica es determinar cuál es el método de posicionamiento de los objetos reales más preciso para vuestro proyecto utilizando estereovisión.

MATERIALES

Para desarrollar esta práctica, descargue los siguientes materiales a una carpeta de trabajo local (en adelante, “[working_folder_path](#)”) en el ordenador donde vaya a ejecutar el cuaderno (*.ipynb) o el fichero Python (*.py).

- PoliformaT -> Recursos/2-Software-&-cuadernos/110-Jupyter lab demo- Stereo Triangularization

Importante >> No olvide hacer una copia de seguridad del código implementado (por ejemplo, en su carpeta W:) al finalizar la práctica.

1.- ARRANQUE CON Jupyter lab

Si nunca has utilizado PyCharm, es mejor empezar con Jupyter lab. Ejecuta el programa Jupyter lab desde un terminal “Anaconda Prompt (miniconda 3)”, tras activar el “environment” adecuado.

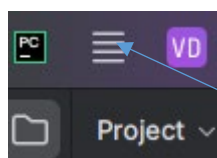
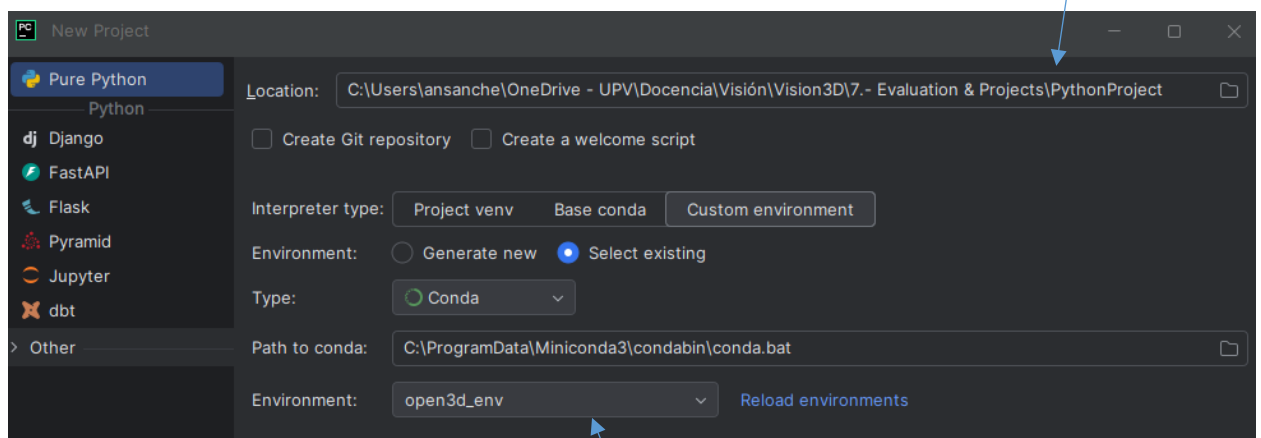
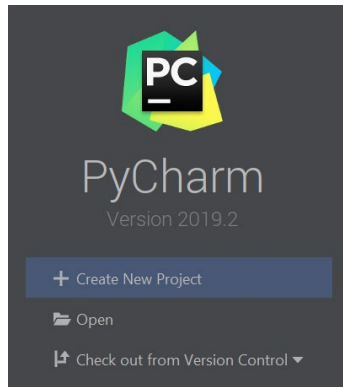
```
(base) C:\windows\system32> conda activate open3d_env
```

```
(open3d_env) C:\Users\username> jupyter lab --notebook-dir “working_folder_path”
```

Utilidad >> Para habilitar el autocompletado de código en JupyterLab, presione la tecla Tab mientras escribe el código.

1.2.- ARRANQUE CON PyCharm

Si prefieres comenzar a trabajar con PyCharm, deberás crear un proyecto y seleccionar el “environment” adecuado desde el desplegable “Environment”.

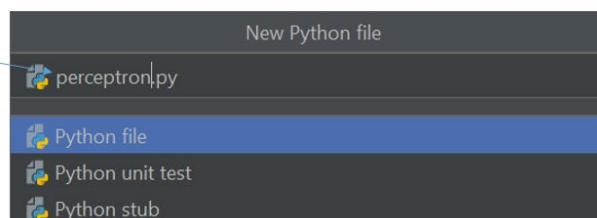


Select environment

Para crear un nuevo fichero Python (*.py) simplemente selecciona la siguiente opción del “Menu principal”:

- Main menú -> File -> New -> Python File

New file name



DEMO 1: TRIANGULARIZACIÓN - CONFIGURACIÓN EN PARALELO

```
##### """Camera class definition"""
class Cam:

    def __init__(self, tX_):
        """Intrinsic parameters"""
        self.fu = 325.62555182 # fu = f/dx
        self.fv = 325.62555182 # fv = f/dy
        self.uo = 330.5 # uo = xo
        self.vo = 187.5 # vo = yo

        """Extrinsic parameters"""
        radius = 6
        angle = 180 - 45
        rX = (pi / 180) * (angle) # Rotation angle on axis X
        rY = 0 # Rotation angle on axis Y
        rZ = pi # Rotation angle on axis Z
        tX = tX_ # Traslation on axis X
        tY = math.sin((pi / 180) * (angle)) * radius # Traslation on axis Y
        tZ = -math.cos((pi / 180) * (angle)) * radius # Traslation on axis Z

        """Construction of the projection matrix"""
        K = np.array([[self.fu, 0, self.uo], [0, self.fv, self.vo], [0, 0, 1]])
        K1 = np.array([[self.fu, 0, self.uo, 0], [0, self.fv, self.vo, 0], [0, 0, 1, 0]])

        Rx = np.array([[1, 0, 0], [0, math.cos(rX), -math.sin(rX)],
                        [0, math.sin(rX), math.cos(rX)]]) # Rotation matrix on axis X
        Ry = np.array([[math.cos(rY), 0, math.sin(rY)], [0, 1, 0],
                        [-math.sin(rY), 0, math.cos(rY)]]) # Rotation matrix on axis Y
        Rz = np.array([[math.cos(rZ), -math.sin(rZ), 0], [math.sin(rZ), math.cos(rZ), 0],
                        [0, 0, 1]]) # Rotation matrix on axis Z
        R = np.dot(np.dot(Rx, Ry), Rz) # Rotation matrix of the camera with respect to the world
        self.T = np.array([[tX], [tY], [tZ]]) # Traslation marix of the camera with respect to the world
        self.wTc = np.concatenate((R, self.T), axis=1), np.array([[0, 0, 0, 1]])
        cTw = np.linalg.inv(self.wTc)

        """Projection matrix"""
        P = np.dot(K1, cTw)
        self.P=P/P[2][3]
        print('Projection matrixx =\n',P)

    # Returns the projections of the wordl points X as pixels U
    def project(self, X):
        U = np.dot(self.P, X)
        U = np.concatenate([[U[0] / U[2]], [U[1] / U[2]], [np.ones(U[0].shape)]]
        return U

    # Shows the image
    def im_show(self, U):
        nPuntos = len(U[0])
        nPuntos_mitad = int(nPuntos/2)
        fig = plt.figure(figsize=(self.uo*2/56,self.vo*2/56))
        plt.gca().invert_yaxis()
        plt.plot(U[0][:nPuntos_mitad], U[1][:nPuntos_mitad], marker='s', ls='');
        plt.plot(U[0][nPuntos_mitad:], U[1][nPuntos_mitad:], marker='^', ls='');

        # Draw image edges as a red frame
        x1 = np.array([0, self.uo*2, self.uo*2, 0, 0])
        y1 = np.array([0, 0., self.vo*2, self.vo*2, 0])
        plt.plot(x1,y1, color='r')

    def get_woc(self): # Get optical center with respect de wordl frame
        return [self.T[0,0], self.T[1,0], self.T[2,0]]

    def get_f(self):
        return self.fu

    def get_wTc(self):
        return self.wTc

    def get_principal_point(self):
        return self.uo, self.vo
```

```

# Projected pixels
U1=cam1.project(Xr)
cam1.im_show(U1)
U2=cam2.project(Xr)
cam2.im_show(U2)

"""Noise introduction in coordinates points"""
ruidoI = 0.0 # Noise added to image points
U1r = U1 + np.concatenate((np.array((np.random.rand(2, nPuntos) - 0.5) * ruidoI),
np.zeros((1, nPuntos)))) # image
U2r = U2 + np.concatenate((np.array((np.random.rand(2, nPuntos) - 0.5) * ruidoI),
np.zeros((1, nPuntos)))) # image

disparity = U1r - U2r
disparity = disparity[0]
print(disparity)

# Base line
wTi = cam1.get_wTc()
iTd = np.linalg.inv(wTi) @ cam2.get_wTc()
b=iTd[0,3]
print('Base line:', b)

# Instanciate cam1 and cam2
cam1 = Cam(0)
cam2 = Cam(-1)

# Reconstruction
iXest = np.ones((4,nPuntos))
iXest[0,:] = (b*(U1r[0]-uo))/disparity
iXest[1,:] = (b*(U1r[1]-vo))/disparity
iXest[2,:] = (b*f)/disparity

wXest = wTi @ iXest
print('Xori:\n', X, '\nXest:\n', wXest)

```

DEMO 2: RECTIFICACIÓN DE IMÁGENES

```

# Rectify the images
def rectify_images(img1, img2, mtx1, dist1, mtx2, dist2, T):
    R = T[:3, :3]
    t = T[:3, 3]

    R1, R2, P1, P2, Q, roi1, roi2 = cv.stereoRectify(mtx1, dist1, mtx2, dist2, img1.shape[:2][::-1], R, t)

    map1x, map1y = cv.initUndistortRectifyMap(mtx1, dist1, R1, P1, img1.shape[:2][::-1], cv.CV_32FC1)
    map2x, map2y = cv.initUndistortRectifyMap(mtx2, dist2, R2, P2, img2.shape[:2][::-1], cv.CV_32FC1)

    img1_rectified = cv.remap(img1, map1x, map1y, cv.INTER_LINEAR)
    img2_rectified = cv.remap(img2, map2x, map2y, cv.INTER_LINEAR)

    return img1_rectified, img2_rectified

```

DEMO 3a: TRIANGULARIZACIÓN - CONFIGURACIÓN GÉNERICA

```
import numpy as np
import matplotlib.pyplot as plt
import cv2 as cv

f, cx, cy = 1000., 320., 240.
pts0 = np.loadtxt('./data/image_formation0.xyz')[:, :2]
pts1 = np.loadtxt('./data/image_formation1.xyz')[:, :2]
output_file = 'triangulation.xyz'

# Estimate relative pose of two view
F, _ = cv.findFundamentalMat(pts0, pts1, cv.FM_8POINT)
K = np.array([[f, 0, cx], [0, f, cy], [0, 0, 1]])
E = K.T @ F @ K
_, R, t, _ = cv.recoverPose(E, pts0, pts1)

# Reconstruct 3D points (triangulation)
P0 = K @ np.eye(3, 4, dtype=np.float32)
Rt = np.hstack((R, t))
P1 = K @ Rt
X = cv.triangulatePoints(P0, P1, pts0.T, pts1.T)
X /= X[3]
X = X.T

# Write the reconstructed 3D points
np.savetxt(output_file, X)

# Visualize the reconstructed 3D points
ax = plt.figure(layout='tight').add_subplot(projection='3d')
ax.plot(X[:, 0], X[:, 1], X[:, 2], 'ro')
ax.set_aspect('equal')
ax.set_xlabel('X [m]')
ax.set_ylabel('Y [m]')
ax.set_zlabel('Z [m]')
ax.grid(True)
plt.show()
```

DEMO 3b: TRIANGULARIZACIÓN – IMPLEMENTACIÓN CONFIGURACIÓN GÉNERICA

```
def triangulatePoints(P0, P1, pts0, pts1):
    Xs = []
    for (p, q) in zip(pts0.T, pts1.T):
        # Solve 'AX = 0'
        A = np.vstack((p[0] * P0[2] - P0[0],
                        p[1] * P0[2] - P0[1],
                        q[0] * P1[2] - P1[0],
                        q[1] * P1[2] - P1[1]))
        _, _, Vt = np.linalg.svd(A, full_matrices=True)
        Xs.append(Vt[-1])
    return np.vstack(Xs).T
```

DEMO 4: RECONSTRUCCIÓN DENSA - CONFIGURACIÓN EN PARALELO

```
"""Estimate the disparity map"""
# Tuned parameters for the 'aloe' image pair
window_size = 3
min_disp = 16
num_disp = 112-min_disp
stereo = cv2.StereoSGBM_create(minDisparity = min_disp,
                               numDisparities = num_disp,
                               blockSize = 16,
                               P1 = 8*3*window_size**2,
                               P2 = 32*3*window_size**2,
                               disp12MaxDiff = 1,
                               uniquenessRatio = 10,
                               speckleWindowSize = 100,
                               speckleRange = 32
)

disp = stereo.compute(imgL, imgR).astype(np.float32) / 16.0

"""Reconstruct world points"""
h, w = imgL.shape[:2]
f = 0.8*w # guess for focal length
Q = np.float32([[1, 0, 0, -0.5*w],
                [0,-1, 0,  0.5*h], # turn points 180 deg around x-axis,
                [0, 0, 0,   -f], # so that y-axis looks up
                [0, 0, 1,    0]])
points = cv2.reprojectImageTo3D(disp, Q)
```

5.-TAREA

A partir de los materiales y ejemplos anteriores deberéis integrar en vuestro proyecto:

- un método de posicionamiento de los objetos reales utilizando un par de estereovisión.